Implementation Document

for

Campus Connect

Version <1.0>

Prepared by

Group 14: **Group Name: <DevAxis>**

Prabhakar Raj	230761	prabhakar23@iitk.ac.in
Navdeep Singh	230684	navdeeps23@iitk.ac.in
Raj	230836	braj23@iitk.ac.in
Dishank Kumawat	230377	dishankk23@iitk.ac.in
Mrityunjay Kumar	230668	mrityunjay23@iitk.ac.in
Prateek Raj	230779	prateekr23@iitk.ac.in
Karan Kumar	230534	karankumar23@iitk.ac.in
Rupam Das	230882	rupamd23@iitk.ac.in
Sumit Vishwakarma	231049	vsumit23@iitk.ac.in
Vatsal Kumar	231126	vastalk23@iitk.ac.in

Course: CS253
Mentor TA: Utkarsh Agrawal
Date: 04-02-2026

CONTENTS II
REVISIONS II

1 IMPLEMENTATION DETAILS 1
2 CODEBASE 4
3 COMPLETENESS 8

APPENDIX A - GROUP LOG 13

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
v1.0	Group 14	The first version of the implementation document	26/03/2026

1 Implementation Details

Our Web Application, CampusConnect, has three major components:

1.1 Backend

1. **Programming Language:** The backend of CampusConnect is built using **Node.js** (JavaScript). Node.js's asynchronous, event-driven architecture makes it highly scalable and perfect for handling real-time features like live discussion forums and socket programming.

2. **Framework:** We used **Express.js**. We chose Express because it is a minimal and flexible Node.js web application framework that provides a robust set of features.

    - **RESTful Routing:** Express makes it simple to set up REST API endpoints for our different modules (Auth, Projects, Forum, Sessions).

    - **Middleware Support:** Allows us to easily implement custom JWT authentication middleware to protect specific routes based on user roles (Admin, Professor, Student).

3. **Libraries and Packages:**

    - **jsonwebtoken (JWT):** Used for securely transmitting information between the client and server as a JSON object, enabling stateless authentication.

    - **Nodemailer / SendGrid:** Used for sending OTPs to user emails during the registration and password reset phases.

    - **Socket.io:** Utilized for native WebSockets programming to power instant, real-time messaging without the latency of HTTP polling.

4. **Authentication & Security:**

    - **Password Handling:** Passwords are securely hashed using bcrypt with a high salt round (10) before being saved to the database.
    - **OTP Verification:** Registration is restricted to @iitk.ac.in emails. A 6-digit OTP is generated and sent to the email. The OTP includes a strict 5-minute time-to-live (TTL) expiration to prevent replay attacks.

- **Data Anonymization:** When a user deletes their account, their credentials and personal data are scrambled (Ghost Protocol), but their unique ID is retained to ensure forum posts and chat histories do not crash the application.

1.2 Frontend

1. **Programming Languages:** The frontend is built using **JavaScript (ES6+), HTML5, and CSS3** within the React ecosystem.

2. **Framework & UI Library:** We used **React.js** due to its component-based architecture, which allows for highly reusable and dynamic UI components.

    - **Tailwind CSS:** Used exclusively for styling. Tailwind's utility-first approach allowed us to rapidly build a modern, responsive UI with custom animations without writing bulky external CSS files.

    - **Lucide-React:** Used for lightweight, consistent, and customizable SVG icons.

    - **React Router DOM:** Handled client-side routing, enabling a fast Single Page Application (SPA) experience without page reloads.

1.3 Database

For the data layer, we utilized **MongoDB Atlas**, a fully managed cloud NoSQL database service.

- **Cloud Reliability:** By using MongoDB Atlas, our database is hosted securely in the cloud, ensuring high availability, automatic backups, and seamless scalability without requiring local database instances.

- **Flexible Schema:** MongoDB easily handles complex embedded arrays (like lists of applicants inside a Project document or upvote arrays in a Forum Post).

- **Relational Mapping via Mongoose:** We used ObjectId references (ref: 'User') to link different data collections (e.g., linking Comments to Posts, and Feedback to Courses), allowing for efficient and powerful .populate() queries.

1.4 Building Systems

- **Vite:** Used as the build tool and development server for the React frontend. Vite offers significantly faster Hot Module Replacement (HMR) and optimized build times.
- **NPM (Node Package Manager):** Used for managing all backend and frontend dependencies safely.

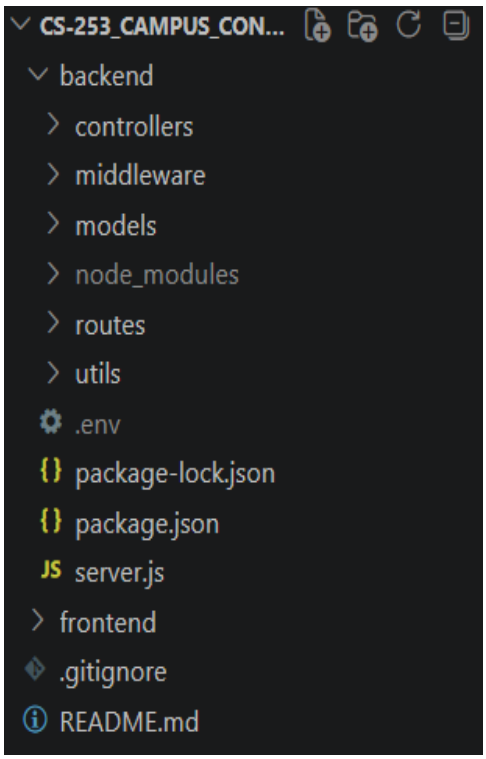
2 Codebase

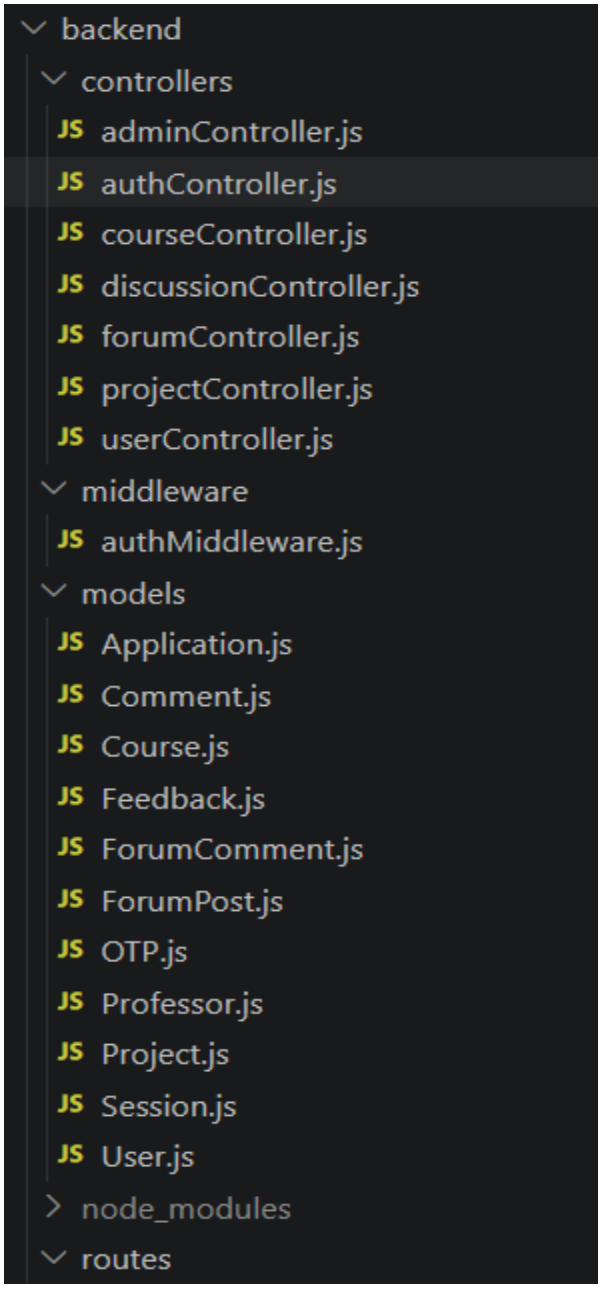
2.1 Github Repository

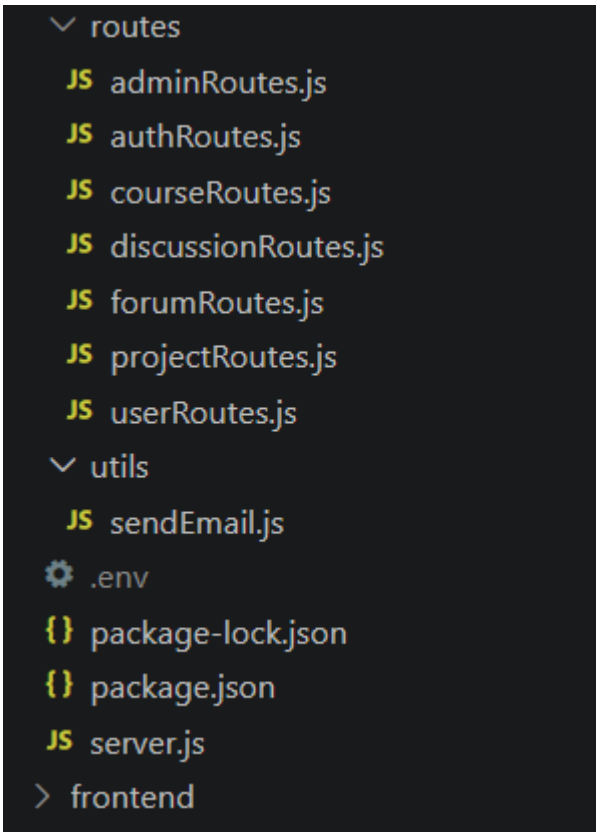
Link: [CS-253_Campus_Connect]

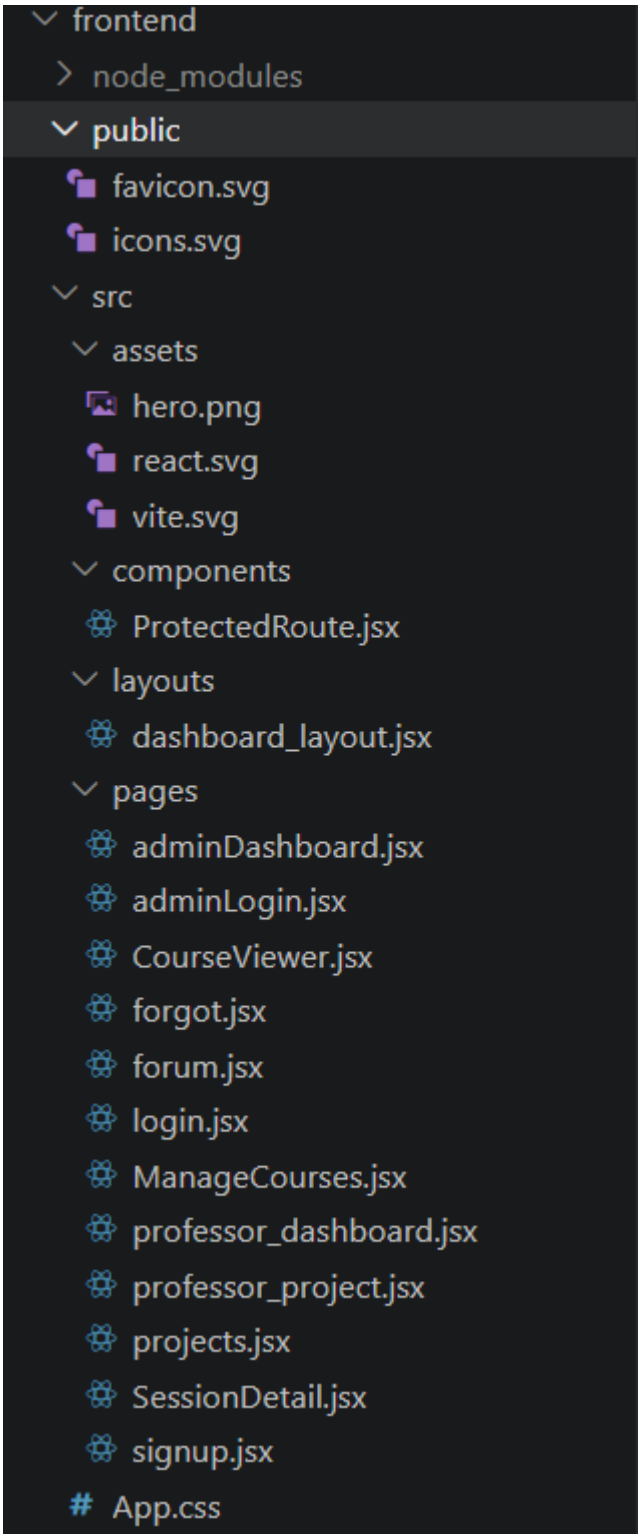
2.2 Codebase Structure

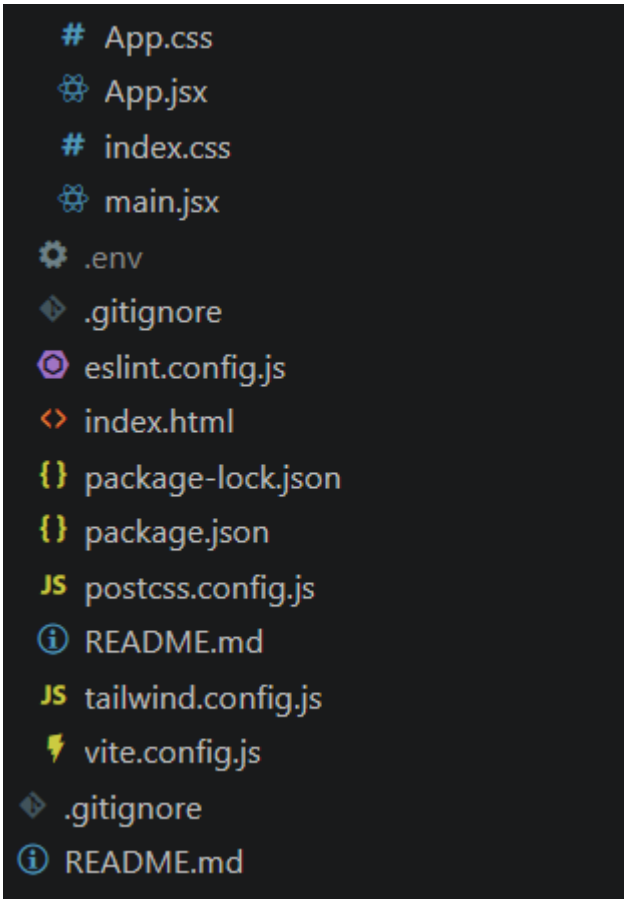
The overall structure of the code is separated into a decoupled frontend and backend:






routes
adminRoutes.js
authRoutes.js
courseRoutes.js
discussionRoutes.js
forumRoutes.js
projectRoutes.js
userRoutes.js
utils
sendEmail.js
.env
package-lock.json
package.json
server.js
frontend


frontend
node_modules
public
favicon.svg
icons.svg
src
assets
hero.png
react.svg
vite.svg
components
ProtectedRoute.jsx
layouts
dashboard_layout.jsx
pages
adminDashboard.jsx
adminLogin.jsx
CourseViewer.jsx
forgot.jsx
forum.jsx
login.jsx
ManageCourses.jsx
professor_dashboard.jsx
professor_project.jsx
projects.jsx
SessionDetail.jsx
signup.jsx
App.css


App.css
App.jsx
index.css
main.jsx
.env
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
postcss.config.js
README.md
tailwind.config.js
vite.config.js
.gitignore
README.md

2.3 Instructions to Setup Development Server

1. **Clone the Repository:**

```
git clone <Repository Link>
cd CampusConnect
```

2. **Setup the Backend:**

```
cd backend
npm install
```

Create a .env file in the backend directory and add your MongoDB Atlas URI and JWT Secret. Start the server using: node server.js

3. **Setup the Frontend:** Open a new terminal window.

```
cd frontend
npm install
npm run dev
```

The application should now be accessible at http://localhost:5173/

3 Completeness

We were able to successfully implement the core features outlined in the SRS document, alongside several advanced additions.

3.1 Sign Up / Login

- **Authentication:** Implemented strict email validation restricting access exclusively to @iitk.ac.in domains to ensure community exclusivity.

- **Role-Based Access:** During registration, users must select between 'Student' and 'Professor'.

- **Secure OTP:** Integrated secure OTP generation and email delivery. Accounts are only created after the user successfully enters the 6-digit code sent to their academic email.

3.2 Admin Panel

- **Platform Management:** We developed a dedicated, highly secure administrative portal accessible via the /admin-login route.

- **Faculty Whitelisting:** Once logged into the admin dashboard, platform administrators possess the sole authority to add and verify Professor email addresses. This strict whitelisting mechanism guarantees that no student can falsely register for a Professor account.

3.3 Workspace & Profile Settings

- **Dynamic Navigation:** Developed a responsive DashboardLayout featuring a dynamic sidebar that automatically adjusts its navigation links based on the user's role (Student vs. Professor).

- **Identity Management:** Integrated a secure Profile Settings modal allowing students to safely change their anonymous usernames (with strict uniqueness checks), while Professors can update their public display names.

- **Security & Deletion:** Users can update their passwords (guided by a real-time UI password strength meter) and execute a secure two-step account deletion process using the cascading "Ghost Protocol."

3.4 Course Management & Schedule

Professors are granted comprehensive control over their academic modules through a dedicated interface.

- **Course Creation:** Instructors can instantly add and define new courses for the semester.

- **Faculty Collaboration:** The system allows the primary professor to add secondary faculty members as "Co-Instructors" to a course.

- **Roster Management:** Professors can manually enroll students into their courses or manage the existing student roster.

- **Course Deletion:** At the end of a semester, or if a course is cancelled, the professor has the administrative rights to safely delete the entire course and its associated data from the system.

3.5 Live Session Discussion (Chat)

- **Real-Time WebSockets:** The live discussion board inside SessionDetail.jsx completely bypasses outdated HTTP short-polling. Instead, it utilizes true **Socket programming** to ensure that messages appear instantly and bidirectionally across all connected clients.

- **WhatsApp-Style UI:** Messages from the current user are aligned to the right, while peer and professor messages are aligned to the left, offering an intuitive chatting experience.

- **Anonymity Rules:** Students participate anonymously (@username) to encourage doubt-asking without hesitation, whereas Professors display their real name accompanied by a distinct "Instructor Mode" badge.

3.6 Session Feedback

- **Evaluations:** We implemented a detailed MCQ feedback form focusing on multi-dimensional metrics (Content Quality, Teaching Delivery, Concept Clarity, Class Engagement, and Lecture Pace).

- **Anonymity & Lock-out:** The feedback is completely anonymous. Once a student submits their review for a specific session, the system locks their state to prevent spam or duplicate submissions.

3.7 Project Board

- **Professor View:** Faculty members can post new research and project openings specifying required skills, minimum CPI, and team size constraints. They can view a structured table of applicants, instantly open attached resumes, and update an applicant's status to Accept or Reject.

- **Student View:** Students can explore available projects, use a dynamic search bar to filter by skill or title, and apply via a cohesive modal form. They can continuously track their live application status via colored UI badges on their dashboard.

3.8 Community Forum

- **Campus Discussions:** Built a robust community board where users can post open questions, seek teammates, or discuss career opportunities.

- **Categorization & Voting:** Posts can be filtered by specific tags (Academics, Projects, Career). We implemented an advanced upvote/downvote algorithm that allows users to cast a vote, switch their vote, or toggle it off seamlessly.

- **Threaded Replies:** The forum supports inline, expandable threaded comments to keep specific academic discussions highly focused and clean.

3.9 Advanced Analytics Dashboard

- **Data Visualization:** Moving beyond basic tables, we have integrated an Advanced Analytics Dashboard for professors.

- **Performance Metrics:** This dashboard aggregates data from the Session Feedback forms, allowing faculty to visualize trends regarding student engagement, teaching delivery scores, and overall course health over the span of a semester.

3.10 Future Implementations

With the core, chat (WebSockets), and analytics features successfully completed, our future development roadmap includes:

- **Automated Email Notifications:** Implementing backend event triggers that send automated emails to users (e.g., notifying a student when a Professor accepts or rejects their project application, or notifying users of a trending forum post).

- **Calendar Integration:** Allowing students to sync their enrolled Course Schedules and Live Session timings directly with Google Calendar or Apple Calendar using .ics file generation.

Appendix A - Group Log

The table below records the major meetings and milestones among group members. Continuous independent development and asynchronous communication occurred outside these recorded sessions.

SL. No.	Date	Timings	Venue	Description
1	15/02/2026	17:00 - 19:00	Hall 12	Initial project planning. Finalized the MERN stack choice. Split team into Frontend and Backend sub-teams.
2	22/02/2026	18:00 - 20:00	Google Meet	Finalized Database Schemas. Configured MongoDB Atlas cloud environment.
3	01/03/2026	14:00 - 18:00	Hall 12	Integrated Auth endpoints. Successfully tested OTP email delivery and the /admin-login whitelisting panel.
4	10/03/2026	15:00 - 18:30	KD Library	Completed the Dashboard Layout UI and integrated the Project Board (Apply/Approve) and Course Management systems.
5	18/03/2026	16:00 - 19:00	Hall 12	Developed the Live Session Chat using Socket programming (WebSockets) and the MCQ feedback system.
6	22/03/2026	14:00 - 17:00	KD Library	Completed the Community Forum with upvote/downvote logic and built out the Advanced Analytics Dashboard.
7	26/03/2026	19:00 - 22:00	Google Meet	Final UI polish. Implemented the "Ghost Protocol" for safe account deletion. Finalized the Implementation Document.